PROJECT PROPOSAL:
Cristina Gutiérrez Alcalá

I have decided to do a Project where I extract and analyse the formants F1 and F2 of different Spanish accents to be able to build multiple vocal spaces. My main idea is to use Mozilla Common Voice (https://mozilladatacollective.com/datasets/cmn4z1n52000knv07h01532dd) because it is open source, and because they have a significant amount of data to work with.

There are many different types of accents, but I think it would be better to look for peninsular dialect variations (like North vs. South) or standard Spanish compared to Mexican. For now, I do not know how many I will actually use. I would like to apply DSP techniques to be able to automatize the phonetic analysis of dialect variations.

Following the course schedule, I am defining my theme in April to start working on the data. From here, my plan for the project is:

- May - June: Work on the Python code to extract the F1 and F2 formants. I will upload regular weekly progress reports (updates) in Ilias and discuss any issues I find with the code during the lectures.
- July: I will present my results in a 20 min talk with slides. I saw the presentation talks are scheduled between July 7th and July 16th.
- October: I will finish and submit the written report of 12+ pages.

05.05.2026 - Project Update

Data extraction

My main problem was that the original dataset was 50 gbs big. It took me two days and loads of trouble shooting to be able to finally download everything because my laptop is very slow, and also I didn't have enough memory in the IMS server. Also, I thought that if I wanted to process all the data in the dataset (which was almost 2 million datapoints) I would run out memory before I could even achieve anything.

The fix was to first extract everything into my desktop from the zip file and then I built a small snippet of code ("preprocesador.py") which reads "validated.tsv" to extract randomly 100 audios from the north and the south variants of Spanish each. This way I could test that everything worked before trying it with the complete dataset.

Extractor

I built another small extractor ("extractor.py") which uses Praat to extract f1 and f2 (for now) from a test .wav file of me saying /a/. It calculates the whole energy curve from the audio and it looks for the maximum intensity peak. My original code cut the audio in hald and the looked for the formants, but if I was phonating a fricative I would get completely wrong data. I assumed that vowels are phonated when the vocal tract is most open, so the

peak helps me get the nucleus of the vowel.

Pipeline

I filter everything into a folder on my computer called “audios_filtrados” and the coded processes the first 200 extracted audios one by one to get the formants. The output is a file called “resultados_formantes.csv” which saves all the information that was extracted. I will attach it in the following days, because I am having troubles with the extraction.

14.05.2026 - Project Update

Vowel labelling

My goal for this session was to add vowel labels (a/e/i/o/u) to the 569 extracted tokens in “results_formants_clean.csv” (291 North, 278 South). I built a script (“etiquetar_vocales.py”) and tried two approaches:

- **v1**: I used acoustic ranges (Quilis 1981) to classify them. My problem was that it labeled almost all /a/ as North and /i/ as South. I realized this is because Quilis’ ranges are calibrated for Northern Spanish. Using F1/F2 to label the data creates circularity.
- **v2**: I tried inferring the vowel from the transcription text instead. This was too conservative (only 14 valid tokens) because the clips are full sentences. Without phonetic alignment, I don’t know exactly what phoneme is sounding during the intensity peak, so this adds too much noise.

Methodology update

The fix was to abandon individual vowel labelling via text. From here, my plan for the analysis is:

- **Global analysis**: Compare the North and South distributions over the whole F1/F2 space using statistical tests (Welch’s, Cohen’s d). The data already shows the South has a lower F1 and higher F2, which matches Andalusian articulation.
- **K-means (k=5)**: I will let the algorithm find five clusters in the data without imposing external ranges. Then I will interpret the centroids as /a/ /e/ /i/ /o/ /u/ depending on their position. This is methodologically much better.

Next steps

- Next session: Run ”analsis.py” review the plots, and interpret the k-means results.

25.05.2026 - Project Update

Bandwidth extraction

Following feedback from Prof. Wokurek, I updated the extractor to also extract B1 and B2 (the bandwidths of F1 and F2) alongside the formants. Bandwidth measures how “sharp” or “diffuse” each formant peak is, a narrow bandwidth means a well-articulated, tense vowel, while a wide one indicates a reduced or more relaxed articulation. This is relevant

for dialect comparison because Andalusian Spanish is known for vowel reduction. The updated script is “extractor_v2.py” and the cleaning script was also updated (“limpieza_v2.py”) to filter implausible bandwidth values. After re-running the full pipeline on the new computer, I ended up with 440 clean tokens (244 North, 196 South).

Analysis results

I ran a full analysis (“analisis_v2.py”) that answered four questions from the professor:

F1, F2, B1, B2 comparison: F1 and F2 are both significantly different between dialects (as before). The most interesting new finding is B2: the South has almost three times the B2 of the North (408 Hz vs 152 Hz, Cohen’s d = 1.387), which is as large an effect as F1. B1 is not significant, which means both dialects articulate vowel height similarly, the difference is in tongue backness and how diffuse that movement is.

K-means by dialect: I ran k-means separately on North and South using the elbow method and silhouette scores to find the optimal k. Both dialects came out with k=3, meaning the data organises into three broad zones (open / mid / closed) rather than five distinct vowels. This makes sense given that I don’t have phonetic alignment.

Bandwidth vs no bandwidth: Adding B1/B2 to the clustering actually lowers the silhouette score (0.370 -> 0.298), which means bandwidth captures a different kind of structure than formants do, they are not redundant dimensions. This is an interesting finding for the report.

Subgroups within the South: I ran k-means freely on just the Southern data in 4D (F1/F2/B1/B2). The optimal k came out as 5, but with low silhouette scores overall (0.26–0.28), so the subgroups are not very clearly separated. Some subgroups have very high B2 values which could point to more extreme Andalusian realisations, but I would need province-level metadata to confirm this.

Next steps

- Prepare the slides for the presentation and start structuring the written report.

01.06.2026 - Project Update

Who actually recorded my clips?

Before preparing the slides I wanted to double check the 440 tokens from the last update. Common Voice has a column called “client_id” that I had never used, and it is the anonymous id of the person who recorded the clip. I merged “results_formants_clean.csv” with “validated.tsv” on the file path to get it, and what I found made me stop everything.

- The 244 Northern tokens come from 54 speakers, but 132 of them (54 %) are from one single person.
- The 196 Southern tokens come from 26 speakers, and 157 of them (80 %) are from one single person.

So my Welch tests were comparing 440 “independent” observations that are not independent at all. In the worst case, what I called a difference between two dialects is a difference between two people. I do not know yet how much of the effect this explains, but I cannot present the results as they are.

Next steps

- Quantify how much of the effect comes from those two speakers before I write anything else.

08.06.2026 - Project Update

Measuring the damage

I wrote a script (“s9_pseudorep.py”) that repeats the midterm analysis and then attacks it in four different ways. The problem has a name, pseudoreplication, which helped me a lot to find literature about it. The results were quite bad:

- **Capping tokens per speaker**: if I allow every speaker only one token, d(F1) drops from +1.335 to +0.105 and d(B2) from −1.433 to −0.145. The effect size grows monotonically with how much one speaker is allowed to dominate, which is exactly what an artifact looks like.
- **Speaker means**: aggregating each speaker to one row gives d(F1) = 0.04 with p = 1.000 after Holm correction. Nothing survives.
- **Mixed models**: with speaker as a random intercept the ICC for F1 is 0.54, which means my 440 clustered tokens carry roughly the information of 128 independent observations.
- **Bootstrap**: resampling tokens gives a confidence interval of [+1.11, +1.60] for d(F1), but resampling whole speakers gives [−0.20, +1.83], which includes zero.

So the three big effects of the 25.05 update are not properties of the dialects. I was honestly a bit demoralised, but my supervisor pointed out that this is a result in itself, and a much more interesting one than the original descriptive claim. I decided to rebuild the whole study with the speaker as the unit of analysis, and to keep the old scripts so that the report can show the before and after.

15.06.2026 - Project Update

New corpus and speaker-aware sampling

First problem to fix: the sample. I downloaded the newest release of Common Voice (26.0, 48 GB) and rewrote “preprocesador.py” as “s1_sample.py”. Extracting the whole tarball again would have taken hours, but the .tsv files sit at the front of the archive, so with tar --fast-read I can pull out “validated.tsv” in about a second and only extract the mp3s I actually need.

What changed in the sampling:

- Every row now carries its client_id, and each clip gets a random but reproducible rank inside its speaker, so I can cap clips per speaker without re-running anything.
- The cap is 8 clips per speaker. The Gini coefficient of clips per speaker went from 0.85 to 0.18 in the South and from 0.76 to 0.26 in the North.
- Speakers who tick more than one region are now dropped. My old code did “norte if norte in x else sur”, which quietly put all of them in the South.
- I added Mexican Spanish as a control group (300 speakers). If my pipeline can separate Peninsular from Mexican but not North from South, then a null result means something about the dialects and not about my method.

The new sample is 5,577 clips from 957 speakers (469 North, 188 South, 300 Mexican). The South only has 188 speakers in the whole corpus, so that number is what limits the entire study.

22.06.2026 - Project Update

Forced alignment

This is the part I was avoiding since the 14.05 update, and it turned out to be the thing that solves it. I installed the Montreal Forced Aligner, which needs its own conda environment because it pulls in Kaldi and a different Python, so my scripts call it from the command line instead of importing it.

“s2_prepare.py” converts every mp3 to 16 kHz mono WAV and writes the transcription next to it as a .lab file. Two details that cost me time: the audios have to be laid out in one folder per speaker, because MFA uses the folder structure to adapt to each voice, and after resampling I have to scale the peak down, because otherwise the samples clip and the distortion ruins the spectrum I want to measure.

“s3_align.py” runs the alignment with the spanish_mfa acoustic model and dictionary. 5,572 of the 5,577 clips aligned (99.9 %); the five that failed have words that are not in the dictionary. Now every clip is a list of phonemes with their start and end times, which means I finally know which vowel I am measuring, without using F1/F2 to guess it. The circularity problem from 14.05 is gone.

29.06.2026 - Project Update

A completely new extractor

With the alignment I could throw away the intensity peak logic and write “s4_extract.py”, which measures every vowel of every clip instead of one frame per clip. Each row now has:

- F1, F2, B1 and B2 measured at 25 %, 50 % and 75 % through the vowel, so I can also see whether the formants move inside the vowel.

- Duration, F0 and HNR, plus the lexical stress of the syllable (Spanish spelling gives you that for free if you know how many syllables the word has, and the aligner tells me).
- The phoneme that comes next, and whether there is an /s/ in coda after the vowel. This one is for a hypothesis I want to test later.

I also fixed something Prof. Wokurek mentioned in the presentation questions: my old code used a 5000 Hz ceiling for the Burg algorithm for everybody, which systematically misestimates female voices. Now the ceiling depends on the speaker (5000 / 5500 Hz), which matters because my dialect groups are not balanced by sex.

Result: 100,053 vowel tokens from 484 minutes of audio, instead of 440.

06.07.2026 - Project Update

Quality control, normalisation and tests

“s5_normalize.py” cleans the tokens and puts the speakers on a common scale. The filters are the same idea as “limpieza_v2.py” but now I can also reject outliers per speaker and per vowel ($z > 3$), because I know which vowel each token is. 89,958 tokens survive (90 %).

Then I add Lobanov z-scores per speaker and Bark values. This is something I did not have before at all: raw Hz mixes the dialect with the length of the vocal tract, so half of my old North/South difference could have been a difference in body size. 906 of the 957 speakers have enough vowels to be normalised.

I also wrote real tests (“tests/test_pipeline.py”). The important one synthesises a vowel with formants that I choose and checks that the extractor gets them back. It found a genuine limitation: linear prediction overestimates F1 for close vowels, +13 % for /i/ and +34 % for /u/, because F1 gets close to F0. It happens equally in both dialects so it cannot create a false difference, but /u/ is now officially my least reliable measurement.

13.07.2026 - Project Update

Presentation and the speaker-level results

I gave the 20 minute talk this week. I presented the pseudoreplication story instead of the original results, which was a bit uncomfortable but went better than I expected.

Meanwhile “s6_stats.py” has the corrected analysis, with 461 Northern and 186 Southern speakers:

- Nothing is significant after Holm correction. Not F1, not F2, not the bandwidths, not duration, not F0, not HNR. The largest effect anywhere is $d = 0.17$ in absolute value.

- Vowel space area (the convex hull of a speaker's five vowels): North 3.05, South 2.95, d = +0.12 with a confidence interval of [−0.05, +0.28]. So no vowel reduction in the South, or at least none this sample can see.
- Power analysis: with these speaker numbers I have 80 % power for d ≥ 0.24. That is what lets me say "no difference bigger than this" instead of just "I found nothing".

One thing that almost fooled me: if I fit a mixed model on Lobanov values token by token, the ICC comes out as 0 and $p < 0.001$, which looks like a beautiful significant result. It is arithmetic, not evidence. Lobanov sets every speaker's mean to 0 and their SD to 1, so there is no between-speaker variance left for the random intercept to absorb and the model goes back to treating tokens as independent. Aggregating to speaker x vowel cells first is the fix.

20.07.2026 - Project Update

Clustering and classification, done properly

"s7_cluster.py" revisits the k-means from 25.05. Now that every token has a real vowel label I can score the clustering with the adjusted Rand index (how much of the five vowels it recovers) instead of only silhouette. Two things came out of this:

- Silhouette still peaks at k = 3, exactly like in the midterm, but ARI peaks at k = 4 and the true number of vowels is 5. So my "three perceptual zones" interpretation was a property of the metric, not of Spanish phonology. Silhouette prefers few compact blobs.
- The bandwidth comparison from 25.05 was not valid either: silhouette falls when you add dimensions no matter how good the clusters are, so comparing 2D against 4D that way says nothing. With ARI, which is comparable across feature sets, bandwidth makes the vowels harder to recover, not easier.

"s8_classify.py" asks the same question as a classification problem: can a model predict the dialect from the acoustics? I run every model twice, once with random folds and once with folds where a speaker is never in training and test at the same time. The gap between the two is the classifier recognising voices instead of dialects, which is the same mistake as before but expressed as a number anyone can read.

27.07.2026 - Project Update

Coda /s/, the hypothesis that does survive

Everything so far is negative, so I wanted to test something concrete that the dialectology literature actually predicts. The best documented feature of Southern Spanish is not the vowels, it is that syllable-final /s/ weakens to an aspiration or disappears. This is only testable with alignment, because I need to know where each /s/ is and whether it is in a coda.

“s10_codas.py” measures the spectral centre of gravity of 21,639 /s/ tokens (12,269 in coda, 9,370 in onset) from 890 speakers. Aspiration moves energy out of the high frequencies, so an aspirated /s/ should have a much lower CoG. The results:

- In coda, the South has a CoG of 2419 Hz against 3065 Hz in the North (d = +0.56). In onset, where the process does not apply, the two are basically the same (d = +0.06). The dialect x position interaction is significant, p = 0.006.
- Using onset minus coda as a weakening index, the South differs from North and Mexico pooled with d = +0.63, p < 0.0001. And the three varieties order exactly as the literature predicts: North +359 Hz, Mexico +836 Hz, South +1022 Hz.
- The vowel before a coda /s/ also shifts (dialect x context on F1, p = 0.005), although the effect is small and the direction needs care: both varieties close the vowel, the South just closes it less.

These survive because they are within-speaker contrasts: each speaker is their own control, so having more clips from one person cannot fake them. That is the real lesson of the whole project, and it is what the report is going to be about.

03.08.2026 - Project Update

Putting the pipeline together

I tidied everything into eleven stages that run in order with a single command (“run_all.py”), so the whole study can be reproduced from scratch in about 15 minutes. Each stage writes its output to disk, which means I can re-run only the statistics without aligning 5,577 clips again. All the paths, the random seed and the thresholds are in one file (“config.py”), so there are no magic numbers spread around the scripts.

“plots.py” makes the report figures. All of them are built from speaker-level values on purpose: if I plot the token clouds the two dialects separate beautifully, because most of the Southern tokens are from one person. That figure would look great and be completely wrong.

I also keep a file (“RESULTS.md”) with every number the report needs and the table it came from, so I do not have to go looking for them while writing.

14.08.2026 - Project Update

Writing the report

The report is written and finished, in the ACL template, and I called it “From Speakers to Dialects”. The structure follows what actually happened during the project: the original analysis, the audit that showed the effects were two speakers, the rebuilt pipeline, the null result at the speaker level, the Mexican control group that shows the method does work, and the coda /s/ result at the end.

Looking back at my proposal from April, the goal was to build vowel spaces for different Spanish accents and I did build them, they just do not differ. The project ended up being about something I did not plan for: what a volunteer corpus does to your statistics if you do not look at who is behind the recordings.

Next steps

- Submit the report, the code, the tables and the figures.
- The code is commented and cleaned up; the audios, the venv and the original .tsv files are not in the zip because they are several GB, but the README explains how to rebuild them from the Common Voice tarball.